MeReader: An Offline, Privacy-Preserving, Progress-Aware AI Assistant for Narrative eBook Reading

Mohammed Efaz[1[0009−0002−3562−7607]], Itilekha Podder[1[0000−0001−7999−5597]], and Udo Bub[1[0000−0002−6018−2411]]

Faculty of Informatics, Eötvös Loránd University (ELTE), Budapest, Hungary
efaz@mohammedefaz.com, itilekha19@inf.elte.hu, udobub@inf.elte.hu

Abstract. This paper introduces MeReader, a privacy-perserving, offline and progress-aware AI assistant designed to function as a contextual companion for narrative reading on consumer hardware. Unlike conventional Large Language Model (LLM)-based systems that process documents/books online in an all-texts-at-once type of way, thereby risking narrative spoilers, MeReader confines both retrieval and generation to the reader's distinct position in the text. The system integrates a hybrid retrieval architecture, combining vector similarity, BM25, and summary embeddings, to ensure high fidelity to the reader's current context. We evaluate the system across classical English books using both quantitative comprehension metrics and qualitative LLM-based independent judging. The results identify a quality-latency frontier, showing that mid-sized local models (2b-4b) can achieve comprehension parity with larger counterparts (5b+) while maintaining acceptable response quality. Importantly, the proposed progress-aware boundary mechanism is shown to prevent future content leakage without compromising retrieval quality and correctness. These findings suggest that a localized, privacy-centric approach can augment, rather than replace, the digital reading experience, offering a good alternative to cloud-dependent services. Therein, there remains a lack of amalgamate systems which combine progress-aware, privacy-preserving, and narrative eBook reading with AI support. It is in this very gap that the development of MeReader came to be; synthesizing said strands into an offline, book-constrained AI assistant, focused on enhancing comprehension without compromising narrative continuity or user privacy.

Keywords: Progress-aware retrieval · RAG · On-device LLM · eBook reader · Privacy-preserving.

1 Introduction

Reading technologies have significantly reshaped the way individuals consume and interact with textual content. While conventional electronic book (eBook) readers have democratized access to literature, they remain largely passive platforms, offering limited support for deeper cognitive engagement beyond basic

features such as highlighting, bookmarking, and keyword search [1]. In parallel, the advent of powerful large language models (LLMs), such as OpenAI's ChatGPT [21] and Meta's LLaMA series [16], has introduced highly interactive paradigms that allow users to upload complete documents and receive artificial intelligence (AI)-generated summaries or responses to natural language queries.

These systems offer efficiency, yet they displace the cognitive effort paramount for critical reading. By giving answers rather than facilitating discovery, they risk reducing readers to passive recipients. LLM-based tools that process entire texts at once often reveal spoilers, disrupting narrative continuity, while traditional eReaders lack support for tracking complex threads. This dual shortfall, AI that does too much and eReaders that do too little, showcases the need for a system that leverages AI to augment, rather than supplant, the reading experience.

In response, this study introduces MeReader, an offline, privacy-preserving, and progress-aware eBook reader designed to resolve the limitations of current market alternatives. While "progress gating" exists in commercial ecosystems like Amazon Kindle's "Ask This Book" [22], it is a closed feature with proprietary formats and cloud services, both of which are antithetical to the philosophy and the aim of MeReader. There exists also, in the realm of PDFs, tools like the Adobe Acrobat AI Assistant [31] that tackle primarily document summarization but their focus is more on productivity rather than narrative eBook reading; treating a novel like a database and querying answers and/or summaries based on prompts without regards to automatic spoiler prevention. Open source alternatives, such as the ever popular KOReader [12] with its AI Assistant plugin [5], prioritize privacy, but rely on a "trailing window" of recent text (limited to 100k characters) which in turn leads to an amnesia, of sorts, in its retrieval and generation system; the inability to recall plot points from early chapters once the reader advances deep in the book. MeReader amalgamates these capabilities (or lack thereof) by generalizing the spoiler-safety of Kindle into an open architecture (one that is auditable) while also maintaining the deep, full-book retrieval memory lacking in the KOReader plugin.

This study aims to advance digital reading technologies by mitigating cognitive constraints associated with text consumption, e.g. reduced spatial memory and diminished recall, through the development of a minimally intrusive, context-sensitive AI assistant. The proposed system aims to maintain narrative coherence and reader agency while improving comprehension through localized, progress-aware AI.

The remainder of this paper is organized as follows. Section 2 reviews related work on AI-augmented reading tools, semantic retrieval architectures, privacy-preserving inference methods, and progress-aware user modeling. Section 3 presents the design and architecture of MeReader, including EPUB parsing, text embedding, and system integration. Section 4 describes the technical implementation and software stack, detailing model optimization, UI/UX considerations, and inference logic. Section 5 reports the results of empirical evaluation across representative reader tasks. Section 6 discusses the broader implications

of the findings and system limitations and concludes with a summary of contributions, design novelties, and avenues for future development.

2 Related Work

Recent advances in LLMs have demonstrated their potential as reading assistants. Chen and Leitch [3] evaluated LLMs such as Claude for academic reading, reporting improvements in comprehension. However, they also highlighted limitations, including the models' tendency to distract and concerns about privacy due to reliance on cloud infrastructure. Kryściński et al. [13] introduced BOOKSUM, showing that LLMs can summarize long-form literary works such as novels and plays. Similarly, Spatharioti et al. [28] found that LLM-enhanced web search improves the efficiency of information retrieval compared to traditional methods, inclusing those for consumer tasks. In the domain of document-level retrieval, Dense Passage Retrieval (DPR) [11] was a significant step toward semantic search by learning dense representations of both queries and documents. However, as Reichman and Heck [25] observed, DPR systems are typically trained on general corpora and are not easily adapted to specific, unseen documents. Improvements such as RocketQA [24], which leverages cross-batch negatives and distillation techniques, enhance retrieval quality but impose computational demands that limit applicability in low-resource or offline settings. Embedding models like nomic-embed-text-v2 [19], built using a Mixture-of-Experts framework and trained on multilingual objectives, provide a balance between expressiveness and efficiency.

RAG has emerged as a robust strategy for combining semantic retrieval with generative inference. The canonical RAG architecture has been formalized and extended in works by Gao et al. [6] and Wang et al. [30], including modular and task-adaptive variants. Vertical applications of RAG have been explored in domains such as medicine [33,2] and education [18,15], typically in multi-document settings. However, user studies such as Levonian et al. [15] indicate that retrieval strictness can conflict with user expectations, more so in educational contexts. Emerging architectures like FLARE [9] introduce dynamic, content-aware retrieval strategies, though they often trade off control for flexibility. Privacy-preserving and local AI systems have gained attention due to increasing concerns over data security and user autonomy. Xu et al. [32] demonstrated that on-device AI is feasible through techniques such as model quantization and runtime optimization. Lightweight infrastructure, including SQLite [29], Qdrant [23], and local inference frameworks like Ollama [20], supports practical deployment of vector search and inference on edge devices. However, as Mireshghallah et al. [17] and Ippolito et al. [8] showed, even pretrained models can leak memorized content, showing the importance of avoiding fine-tuning on user data. Similar privacy concerns were identified in on-device audio systems by Zhou et al. [34], who advocated for strictly offline processing to protect user interactions.

In summary, existing research has explored AI-assisted reading, semantic retrieval, RAG frameworks, and private inference independently. However, there remains a lack of integrated systems that combine progress-aware, privacy-preserving, and document-specific AI support within a single-reader context. This gap motivates the development of MeReader, which synthesizes these strands into an offline, book-constrained assistant focused on enhancing comprehension without compromising narrative continuity or user privacy.

3 Proposed System Design

MeReader follows a component-based architecture focusing on separation of concerns and modularity. This approach allows individual components of the system to be developed, tested, and maintained independently, thereby reducing the risk of introducing regressions or conflicts in the core functionality during system evolution. It also facilitates easier scaling and integration of new features, ensuring that future enhancements can be incorporated with minimal disruption to the existing codebase. The design of MeReader integrates principles from both object-oriented programming (OOP) and service-oriented (SOA) architecture to establish robust and well-defined boundaries between system components.

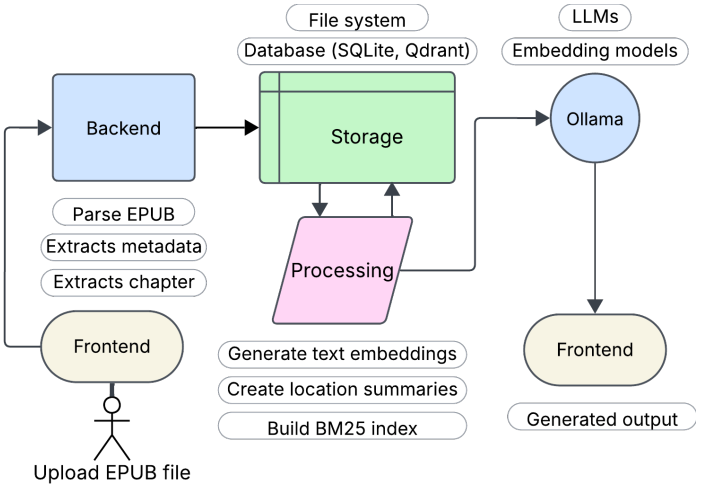
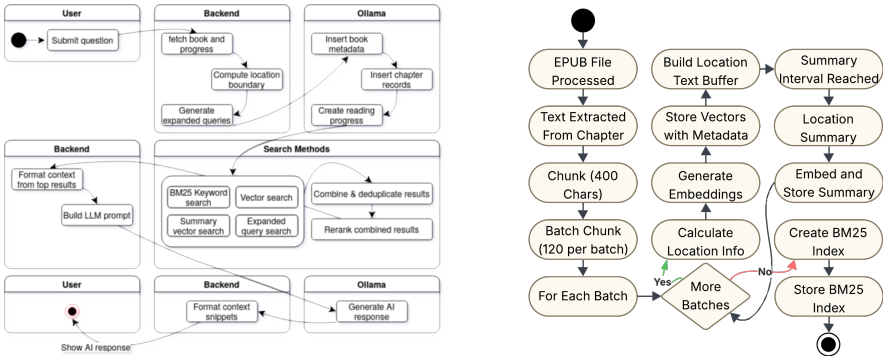


Fig. 1: High-level architecture of the proposed system

The architecture, as shown in Figure 1, is structured to promote clarity, with emphasis on maintaining distinct boundaries between components, thereby supporting modularity and ease of maintenance. The frontend of MeReader is developed using Tauri and Vue.js/JavaScript, leveraging a modular component structure to ensure a responsive and dynamic user experience. Each component is designed to handle a specific aspect of the user interface or interaction logic. The backend of MeReader is implemented using Python with FastAPI, providing a set of RESTful API endpoints that support the application's core function-



(a) AI query flow (b) Embedding pipeline

Fig. 2: System architecture overview: (a) end-to-end AI query flow and (b) embedding and retrieval pipeline.

alities. The backend is organized into specialized services, each responsible for distinct aspects of data processing, content management, and AI-driven features.

MeReader utilizes a variety of storage solutions tailored to the specific needs of different data types within the system. Each storage component is selected to optimize performance, reliability, and scalability for its designated role. Figure 1 also shows the journey of a book when it's first uploaded via the reader. When a user uploads an EPUB file, (1) the original file is saved in a folder, (2) metadata is extracted, (3) content is processed and organized into chapters, (4) database records are created for the book and its structure, (5) text chunks are embedded and stored in the vector database and finally (6) content is made available for reading and querying.

Figure 2a depicts the end-to-end workflow for processing an AI query within MeReader. Upon query initiation, the system first retrieves the relevant book together with the user's current reading position. To ensure contextual focus, a location boundary is then applied, restricting the search space to the most relevant sections of the text. Within this scope, multiple retrieval techniques are executed in parallel, including vector search, BM25 keyword search [26], and query expansion. The outputs of these methods are subsequently consolidated, with duplicates removed and results reranked. The highest-ranked passages are assembled into a contextual block, which is combined with the original query to construct the final prompt. This prompt is submitted to the LLM, whose response is returned to the user alongside the contextual information.

The embedding pipeline, as shown in Figure 2b, converts a given book text into vector representations for semantic search. Important steps include optimal chunking, metadata storing and batched processing, all of which elevate the quality of said embedding. Text flows through sequential processing stages: content extraction, chunking, batch organization, embedding generation, and vector storage. The pipeline also generates summary embeddings at regular intervals to enable higher-level content understanding and builds BM25 indices alongside vector embeddings to support keyword-based search capabilities.

4 Experimental Setup

We evaluated the MeReader architecture described in the previous section to assess its performance across content ingestion, query processing, and retrieval accuracy. The experiments were conducted on the deployed frontend–backend system, with evaluation focusing on component performance and end-to-end user experience.

The storage layer of MeReader employs a two-tier database strategy tailored to heterogeneous data access patterns; SQLite (via SQLAlchemy) for structured metadata and Qdrant for vector embeddings. This setup was benchmarked for scalability with respect to book size, embedding volume, and query throughput. The SQLite schema is normalized for reading applications and uses universally unique identifiers (UUIDs) as primary keys (PK). Foreign keys (FK) maintain referential integrity between entities. Let $book \in \mathsf{Book}$ with attributes (id [PK], metadata, file_ref, progress_attr), $progress \in \mathsf{ReadingProgress}$ with attributes (id [PK], book_id [FK], position, percentage, timestamp), and $chapter \in \mathsf{Chapter}$ with attributes (id [PK], book_id [FK], order, content_path, location_bounds).

Constraints:

1. Each reading progress entry links to exactly one book (1:1):

$$\forall progress,\ \exists! book:\ progress.book_id = book.id$$

2. Each chapter links to one book (1:*):

$$\forall chapter,\ \exists book:\ chapter.book_id = book.id$$

3. On book deletion, associated reading progress entries and chapters are removed (cascade):

$$\text{DELETE Book} \rightarrow \text{DELETE}\{\mathsf{ReadingProgress},\ \mathsf{Chapter}\}$$

The core **Book** entity contains bibliographic metadata including publication details and cataloging identifiers, file system references for content and cover resources, and reading-specific attributes for progress calculation. The **ReadingProgress** entity metadata maintains user state through current position tracking, completion percentages, and temporal access records. The **Chapter** entity provides hierarchical content organization with sequential ordering and location boundaries defining chapter extents.

Qdrant provides specialized vector storage with cosine distance similarity for efficient semantic search operations. Each book maintains a discrete collection enabling progress-aware filtering, with 768-dimensional vectors tagged by location identifiers for exact retrieval boundary enforcement.

Let $query_vector$ be the embedding of the user query text, $filters$ the key–value constraints (e.g., book ID, location boundary), $candidates$ the set of retrieved segments matching the query and filters, $results$ the ordered list of candidates ranked by similarity score, and $top_k(\cdot)$ the function returning the k highest-scoring results.

Vector Search Process:

1. $query_vector = embed_text(user_query)$
2. $filters = \{book_id : target_book,\ location \leq current_position\}$
3. $candidates = similarity_search(query_vector,\ filters,\ threshold)$
4. $results = rank_by_cosine_distance(candidates)$
5. **return** $top_k(results,\ limit)$

The implementation achieves efficient similarity search while maintaining high recall rates for semantic queries through configurable score thresholds (0.6 for content retrieval, 0.65 for summary retrieval).

The storage architecture demonstrates efficient resource utilization. The system processes content using configurable parameters of 400-character chunks with 100-character overlap, optimized for semantic coherence. It comprises multiple components, including raw text content preservation for original document integrity, 768-dimensional vector embeddings for semantic similarity search, BM25 inverted indices for keyword-based retrieval, and relational metadata for progress tracking and content organization. This hybrid approach achieves efficient resource usage, enabling scalable personal library management on consumer hardware while maintaining responsive query performance.

Algorithm 1 Semantic Boundary Detection

Require: HTML content file F, $chunk_size$, $overlap$, min_size
Ensure: Semantically segmented text chunks C

```
text ← ExtractTextFromHTML(F)
position ← 0;   chunks ← []
while position < Length(text) do
    buffer ← text[position : position + 3 × chunk_size]
    chunk_end ← min(chunk_size, Length(buffer))
                                        ▷ Priority 1: paragraph boundaries
    paragraph_break ← RFind(buffer, "\n\n", chunk_size/2, chunk_size+50)
    if paragraph_break ≠ −1 ∧ paragraph_break > chunk_size/2 then
        chunk_end ← paragraph_break + 2
    else                                ▷ Priority 2: sentence boundaries
        sentence_break ← RFind(buffer, ". ", chunk_size/2, chunk_size + 30)
        if sentence_break ≠ −1 ∧ sentence_break > chunk_size/2 then
            chunk_end ← sentence_break + 2
        end if
    end if
    chunk ← Trim( buffer[0 : chunk_end] )
    if Length(chunk) ≥ min_size then
        Append(chunks, chunk)
    end if
    advance ← max(chunk_end − overlap/2, chunk_size/2)
    position ← position + advance
    if position mod (20 × chunk_size) = 0 then
        GarbageCollect
    end if
end while
return chunks
```

The content processing subsystem transforms uploaded EPUB files into searchable, semantically indexed representations while preserving reading progress boundaries essential for spoiler prevention (Algorithm 1). The implementation employs adaptive text segmentation algorithms that prioritize natural language boundaries over fixed-size partitioning.
Let F be the HTML content file, $chunk_size = 400$ the target segment length, $overlap = 100$ the overlap between consecutive segments, and $min_size = 100$ the minimum allowable segment length.

The system implements location tracking using character-based calculations. Each location unit represents 1000 characters, configurable via settings and enabling progress boundaries.

Algorithm 2 Progress-Aware Boundary Enforcement

```
function CalculateSafeBoundary(current_pos, total_pos)
    if current_pos ≤ 0 then
        return 1
    else if current_pos > total_pos then
        return total_pos
    else
        return current_pos
    end if
end function
```

As shown in Algorithm 2, the system calculates safe reading boundaries to prevent spoilers by ensuring users only access content they have already read. The boundary calculation function handles edge cases where users might be at the beginning or end of a book, ensuring valid location ranges for content filtering.
Completion percentage is computed as:

$$progress_ratio \leftarrow \min\Big(100,\ \max\big(0,\ \frac{current_position}{total_positions} \times 100\big)\Big)$$

Spoiler prevention is implemented via Qdrant filtering, e.g. `filter_conditions.append(FieldCondition(key="location",range=Range(lte=boundary)))`, which restricts results to content within the user's current reading progress.

The system appends location based filters during vector searches, restricting results to current progress. The embedding pipeline processes maximum 120 chunks per batch with garbage collection every 20 chunks, generating summaries at 11 location intervals. The RAG architecture employs searches combining vector similarity, keyword matching, query expansion, and progress boundaries, using sequential execution with score normalization (Algorithm 3) and fusion. Alternative queries from local LLMs complement retrieval (lines 20–26), with the `DeduplicateAndRank` function removing semantically redundant results and orders the remaining items by their aggregated relevance score (line 28). Results exceeding eight items where queries contain more than three terms undergo LLM-based reranking on a ten-point scale.

Algorithm 3 Multi-Modal Retrieval with Progress Filtering

Require: Query q, book ID B, current location L, total locations T
Ensure: Ranked context passages R

```
location_boundary ← CalculateBoundary(L, T)
query_embedding ← EmbedText(q)
expanded_queries ←
    GenerateAlternatives(q, book_title)
results ← []
                                    ▷ Vector search (limit = 15, threshold = 0.6)
vector_results ← QdrantSearch(query_embedding, B,
15, 0.6, location_boundary)
for all result ∈ vector_results do
    result.method ← "vector"
end for
Extend(results, vector_results)
                                    ▷ BM25 keyword search (limit = 10)
bm25_results ←
    BM25Search(q, B, 10, location_boundary)
for all result ∈ bm25_results do
    result.method ← "bm25"
end for
Extend(results, bm25_results)
                                    ▷ Summary search (limit = 5, threshold = 0.65)
summary_results ←
    QdrantSearch(query_embedding, B, 5, 0.65, location_boundary, "summary")
for all result ∈ summary_results do
    result.method ← "summary"
end for
Extend(results, summary_results)
                                    ▷ Expanded query search
for all expanded_q ∈ expanded_queries do
    expanded_embedding ← EmbedText(expanded_q)
    expanded_results ←
    QdrantSearch(expanded_embedding, B, 5, 0.5,location_boundary)
    for all result ∈ expanded_results do
        result.method ← "expanded_vector"
    end for
    Extend(results, expanded_results)
end for
return DeduplicateAndRank(results)
```

To assess each component's contribution, we conducted an ablation study by progressively enabling lexical (BM25), dense vector, summary-based, and query-expansion retrieval, evaluating all variants under identical prompts and metrics. Fusion weights were tuned via grid search on a held-out development set to maximize BERTScore F1 under a latency constraint. The selected configuration $w_{\text{bm25}} = 0.40$, $w_{\text{vector}} = 0.60$, $w_{\text{summary}} = 0.40$, and $w_{\text{expanded}} = 0.70$ was robust.

These weights are applied after per-method score normalization to balance the contribution of keyword matching, semantic similarity, summary retrieval, and query expansion under a single ranking signal. Each method's raw scores are normalized by its maximum observed score for the current query, and the resulting normalized scores are then scaled by the corresponding w_{method} before being merged into a combined pool.

The final ranking is produced by sorting the merged candidates by their weighted normalized score, followed by the deduplication pass. In this last step, `DeduplicateByContentSimilarity` removes semantically redundant passages whose overlap exceeds a predefined threshold, ensuring that only distinct context segments are retained while preserving the highest-scoring representative.

For model integration, MeReader leverages local language models via Ollama to enable privacy-preserving and offline operation. Communication occurs through asynchronous HTTP API requests (default endpoint: `localhost:11434`), with configurable support for different model architectures. The completion API exposes standard parameters such as temperature (default 0.7), maximum tokens, and streaming options, while the embedding API uses `nomic-embed-text` to generate 768-dimensional vectors with batched execution.

5 Results and Discussion

Code 1: Comprehension example

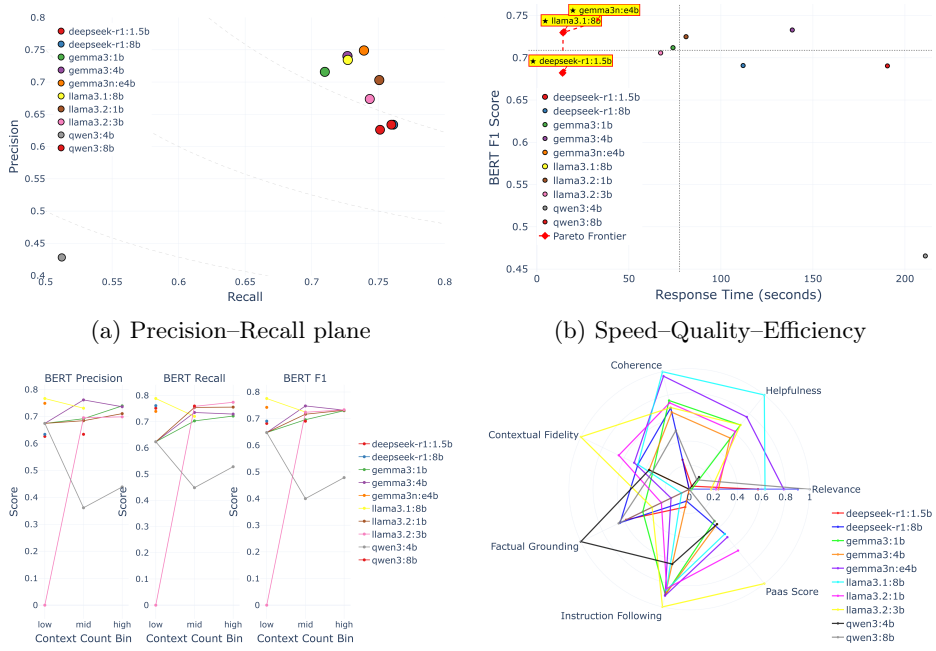
```
{
  "book": "1984",
  "query": "What are the three slogans of
  ↪  the Party?",
  "ground_truth": "WAR IS PEACE, FREEDOM IS
  ↪  SLAVERY, IGNORANCE IS STRENGTH"
}
```

Code 2: Spoiler-prevention example

```
{
  "book": "1984",
  "query": "What happens to the glass
  ↪  paperweight Winston bought?",
  "percentage_of_book_read": 55,
  "evaluation_rules": {
    "forbidden_keywords": ["smashed","piece
    ↪  s","hearth-stone","fragment of
    ↪  coral"]
  }
}
```

We evaluated a total of 600 queries across three literary novels, (i) 1984 (George Orwell) [7], (ii) Of Mice and Men (John Steinbeck) [10], and (iii) The Death of Ivan Ilych (Leo Tolstoy) [14], using both technical and LLM-judged metrics. While this selection provides a focused proof-of-concept for narrative fiction, we acknowledge the constraints: the sample spans classic English literature exclusively, omitting multilingual texts, non-fiction genres, and modern literary forms. These boundaries were deliberate for initial validation, yet they circumscribe the generalizability of our findings. Future iterations will incorporate multi-genre books, literature, and long form novels across languages, to stress test the progress-aware mechanism under diverse linguistic conditions.

The benchmark included 300 comprehension queries (30 queries per model across 10 models) in the format as shown in Code 1, and 300 spoiler prevention scenarios (30 queries per model across 10 models), in the format shown in Code 2. Each comprehension item paired a short reference answer generated with `Gemini`



(a) Precision–Recall plane

(b) Speed–Quality–Efficiency

(c) Reading progress robustness

(d) Holistic quality radar

Fig. 3: Comprehensive evaluation of the system across four complementary perspectives: (a) precision–recall trade-off, (b) speed–quality–efficiency balance, (c) robustness of reading progress under varying conditions, and (d) holistic qualitative assessment using a radar plot.

`2.5 Pro`, while spoiler tasks defined "forbidden facts" with progress-based thresholds. For every item, passages were retrieved through hybrid lexical–semantic search, deduplicated, and limited to six excerpts. The models were instructed to answer only from the excerpts and to respond "Info not available in text so far" when necessary. Responses were generated with temperature 0.3.

Technical evaluation employed BERTScore precision, recall, and F1 to measure semantic similarity with reference answers. As shown in Fig. 3a, the upper-right cluster contains the most reliable small- and medium-sized models, with precision between 0.60–0.75 and recall between 0.70–0.76. `gemma3n:e4b` achieved the highest precision (0.75) with recall 0.74. `Gemma3:4b` and `llama3.1:8b` exhibited similar performance, while `llama3.1:1b` and `llama3.2:3b` favored recall over precision. `Deepseek-r1:1.5b`, `deepseek-r1:8b`, and `qwen3:8b` performed moderately (precision 0.62–0.64, recall $\approx$ 0.75). `Qwen3:4b` was a clear outlier with substantially lower scores (precision 0.43, recall 0.51).

Fig. 3b relates BERT F1 to query response time. Dashed guides indicate a latency threshold ($\sim$ 80s) and a quality bar (F1 $\sim$ 0.71). The target zone lies in the upper-left quadrant. `gemma3n:e4b` and `llama3.1:8b` were Pareto-optimal [27], achieving strong accuracy (F1 0.74–0.75) with moderate latency ($\sim$ 30s). `deepseek-r1:1.5b` was the fastest (15–20s) but less accurate (F1

0.69–0.70). `gemma3:4b` reached F1 $\sim$ 0.73 at much higher latency ($\sim$ 145s). In contrast, `qwen3:8b` and `deepseek-r1:8b` fell below both thresholds, while `qwen3:4b` again underperformed. Overall, `gemma3n:e4b` and `llama3.1:8b` provided the best balance, with `deepseek-r1:1.5b` offering a low-latency alternative when moderate accuracy is acceptable.

Fig. 3c shows performance across progress bins: Low (1–2 passages), Mid (3–4), and High (5–6). Missing markers indicate that no items fell into a given bin due to either retrieval limitations or runner/API errors. `gemma3n:e4b`, `gemma3:4b`, and `llama3.1:8b` remained stable across bins, with precision 0.73–0.77, recall 0.71–0.76, and F1 0.73–0.75. These results suggest that even with minimal evidence, the models produced reliable answers, and additional passages did not degrade quality. `deepseek-r1:1.5b` and `deepseek-r1:8b` tracked slightly below but consistently across bins. By contrast, `qwen3:4b` showed a sharp decline at Mid (F1 $\sim$ 0.40), recovering only partially at High (F1 0.47–0.52). `llama3.2:3b` produced no Low bin results). But, when data was available, its F1 aligned with the $\sim$ 0.74 band. Even with limited evidence (1–2 passages), models maintained stable F1, reflecting adherence to the instruction not to speculate.

For qualitative evaluation, we employed an LLM-as-judge framework (Fig. 3d) across five dimensions: contextual fidelity, relevance, helpfulness, coherence, and instruction following. This approach provided a human-like assessment of response quality while ensuring evaluation consistency. The radar plots reveal that `gemma3n:e4b` and `llama3.1:8b` achieve consistently high scores across most dimensions, especially coherence and contextual fidelity, aligning with their strong precision–recall performance. `deepseek-r1:1.5b` delivers high instruction following but lags in coherence and helpfulness, reflecting its lower BERT F1 despite faster responses. In contrast, `qwen3:4b` underperforms across all axes, particularly in factual grounding and coherence, consistent with its outlier status in quantitative metrics. These findings inform that models with balanced technical accuracy also tend to produce more user-satisfactory outputs in qualitative dimensions.

Fig. 3d summarizes the judgments averaged over the benchmark dimensions. `Llama3.1:8b` and `gemma3n:e4b` form the best overall shapes with high helpfulness, coherence, and instruction following and competing contextual fidelity. `Llama3.2:3b` has a high instruction following and contextual fidelity (strong adherence to prompts and book grounding), but is weaker on relevance (consistent with occasional over-literal responses). `Deepseek-r1:8b` scores high on relevance and factual grounding, but trials on helpfulness. `Qwen3:4b` shows high factual grounding but an overall smaller polygon all around, matching its error profile and low precisions seen in the earlier diagrams. Overall, these readings align with those of the quantitative metrics. Models that are coherent and helpful also sit near the precision-recall frontier, while models with generation errors have visibly collapsed polygons.

6 Discussion, Conclusion and Future Work

The paper introduced MeReader, an offline, privacy-preserving, and progress-aware reading companion that constrains both retrieval and generation to a reader's current position in the narrative. Rather than answering "about the book," it recalls and clarifies without revealing future content or substituting for the act of reading. Empirically, across three novels and 600 items (300 comprehension, 300 spoiler checks), we observe a consistent quality-latency frontier (Figure 3b); `gemma3n:e4b` and `llama3.1:8b` achieve the best balance (Bert F1 $\approx$ 0.74 -0.75 at $\approx$ 30s/item, while `deepseek-r1:1.5b` offers a low-latency alternative ($\approx$ 15-20s at slightly lower quality (F1 $\approx$ 0.69-0.70). Precision-recall distributions (Figure 3a) place `gemma3n:e4b` at the upper frontier (P$\approx$ 0.75, R$\approx$ 0.74) with `llama3.1:8b` and `gemma3:4b` close behind. Robustness vs. context availability (Figure 3c) shows stable performance even with 1-2 passages, indicating the system's retrieval pipeline uses evidence that is both compact and sufficient. Qualitative judging (Figure 3d) aligns with its quantitative counterparts; the Pareto-efficient models also lead on coherence, contextual fidelity, and helpfulness, i.e., they *feel* better to use, not just score better.

The central novelty is to treat spoiler prevention as a first-class citizen, not an after-the-fact filter. MeReader's progress gates ensure that generation cannot cite beyond what the reader has already read. Taken together, the results provide a blueprint for a local, trustworthy reading support on commodity hardware by (1) keeping data and inference on device, (2) incorporating progress metadata into both retrieval and prompting, and (3) targeting frontier-efficient models that deliver strong comprehension at a tolerable latency. For readers, this design encourages active engagement (asking, checking, connecting) instead of passive consumption.

Our study is limited to three English novels, ten small/medium models, and a single workstation. Metrics based on BERTScore and an LLM-as-judge introduce known biases and may not fully capture long-range reasoning. Reference answers generated by `Gemini 2.5 Pro` were concise and may under-reward faithful paraphrasing, and the spoiler protocol captures explicit but not implicit foreshadowing.

For future work, we see four directions to improve upon. First, scale evaluation by testing more models on a wider set of prompts/questions and a larger, more diverse pool of books spanning languages and genres; a broader dataset will enable stronger conclusions about MeReader's effectiveness. Second, broaden generality and accessibility by extending support to multilingual texts and non-fiction. Third, conduct user studies to compare MeReader against baselines, measuring learning outcomes, enjoyment, and engagement. Finally, strengthen evaluation methodology by replacing the judge-as-LLM with human audits.

MeReader shows that progress-aware RAG can deliver spoiler-safe, high-quality reading guidance without leaving the device. By aligning retrieval, prompting, and evaluation with the reader's position, the system augments, rather than replaces, the act of reading. We view this as a practical step towards local AI companions for literary engagement and an invitation to the community to re-

fine the blueprint along whatever axes they see fit. Source code and datasets are available at [4] (MIT License).

References

1. Baron, N.S.: Reading in a digital age. Kappan Online, accessed: May 29, 2025. [Online]. Available: https://kappanonline.org/reading-digital-age/
2. Cabello, L., Martin-Turrero, C., Akujuobi, U., Søgaard, A., Bobed, C.: Meg: Medical knowledge-augmented large language models for question answering. arXiv preprint arXiv:2411.03883 (2025). https://doi.org/10.48550/arXiv.2411.03883
3. Chen, C., Leitch, A.: Llms as academic reading companions: Extending hci through synthetic personae. arXiv preprint arXiv:2403.19506 (2024). https://doi.org/10.48550/arXiv.2403.19506
4. Efaz, M.: MeReader: Progress-Aware, On-Device Reading Companion. GitHub repository (2025), https://github.com/WhiteHades/mereader
5. Faruq, O.: Assistant: AI helper plugin for KOReader. https://github.com/omer-faruq/assistant.koplugin (2026), gitHub repository
6. Gao, Y., et al.: Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997 (2024). https://doi.org/10.48550/arXiv.2312.10997
7. George Orwell: Nineteen eighty-four, accessed: Nov. 04, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Nineteen_Eighty-Four
8. Ippolito, D., et al.: Preventing verbatim memorization in language models gives a false sense of privacy. arXiv preprint arXiv:2210.17546 (2023). https://doi.org/10.48550/arXiv.2210.17546
9. Jiang, Z., et al.: Active retrieval augmented generation. arXiv preprint arXiv:2305.06983 (2023). https://doi.org/10.48550/arXiv.2305.06983
10. John Steinbeck: Of mice and men, accessed: Nov. 04, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Of_Mice_and_Men
11. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2004.04906 (2020). https://doi.org/10.48550/arXiv.2004.04906
12. KOReader Contributors: KOReader: An ebook reader application supporting pdf, djvu, epub, fb2 and many more formats. https://github.com/koreader/koreader (2026), gitHub repository
13. Kryściński, W., Rajani, N., Agarwal, D., Xiong, C., Radev, D.: Booksum: A collection of datasets for long-form narrative summarization. arXiv preprint arXiv:2105.08209 (2022). https://doi.org/10.48550/arXiv.2105.08209
14. Leo Tolstoy: The death of ivan ilyich, accessed: Nov. 04, 2025. [Online]. Available: https://en.wikipedia.org/wiki/The_Death_of_Ivan_Ilyich
15. Levonian, Z., et al.: Retrieval-augmented generation to improve math question-answering: Trade-offs between groundedness and human preference. arXiv preprint arXiv:2310.03184 (2023). https://doi.org/10.48550/arXiv.2310.03184
16. Meta: Llama, accessed: May 29, 2025. [Online]. Available: https://www.llama.com/
17. Mireshghallah, F., Uniyal, A., Wang, T., Evans, D., Berg-Kirkpatrick, T.: An empirical analysis of memorization in fine-tuned autoregressive language models. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 1816–1826 (2022). https://doi.org/10.18653/v1/2022.emnlp-main.119

18. Mitra, C., Miroyan, M., Jain, R., Kumud, V., Ranade, G., Norouzi, N.: Retllm-e: Retrieval-prompt strategy for question-answering on student discussion forums. Proceedings of the AAAI Conference on Artificial Intelligence **38**(21) (2024). https://doi.org/10.1609/aaai.v38i21.30368
19. Nomic AI: Nomic Embed Text V2: An Open Source, Multilingual, Mixture-of-Experts Embedding Model, accessed: Jun. 04, 2025. [Online]. Available: https://www.nomic.ai/blog/posts/nomic-embed-text-v2
20. Ollama: Ollama, accessed: Jun. 04, 2025. [Online]. Available: https://ollama.com
21. OpenAI: ChatGPT, accessed: May 29, 2025. [Online]. Available: https://openai.com/chatgpt/overview/
22. Preston, D.: Kindle app now answers questions about the book you're reading. The Verge (Dec 2025), https://www.theverge.com/news/844538/kindle-app-ask-this-book-ai-ios
23. Qdrant: Qdrant - Vector Database, accessed: Jun. 04, 2025. [Online]. Available: https://qdrant.tech/
24. Qu, Y., et al.: Rocketqa: An optimized training approach to dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2010.08191 (2021). https://doi.org/10.48550/arXiv.2010.08191
25. Reichman, B., Heck, L.: Dense passage retrieval: Is it retrieving? arXiv preprint arXiv:2402.11035 (2024). https://doi.org/10.48550/arXiv.2402.11035
26. Robertson, S., Zaragoza, H., Taylor, M.: Simple bm25 extension to multiple weighted fields. In: Proceedings of the thirteenth ACM international conference on Information and knowledge management. pp. 42–49 (2004)
27. ScienceDirect: Pareto frontier - an overview, accessed: Sep. 10, 2025. [Online]. Available: https://www.sciencedirect.com/topics/engineering/pareto-frontier
28. Spatharioti, S.E., Rothschild, D.M., Goldstein, D.G., Hofman, J.M.: Comparing traditional and llm-based search for consumer choice: A randomized experiment. arXiv preprint arXiv:2307.03744 (2023). https://doi.org/10.48550/arXiv.2307.03744
29. SQLite: SQLite Home Page, accessed: Jun. 04, 2025. [Online]. Available: https://www.sqlite.org/
30. Wang, X., et al.: Searching for best practices in retrieval-augmented generation. arXiv preprint arXiv:2407.01219 (2024)
31. Weatherbed, J.: Adobe Acrobat gets a generative AI assistant to chat with your documents. The Verge (Feb 2024), https://www.theverge.com/2024/2/20/24077217/adobe-acrobat-generative-ai-assistant-chatbot-pdf-document
32. Xu, J., et al.: On-device language models: A comprehensive review. arXiv preprint arXiv:2409.00088 (2024). https://doi.org/10.48550/arXiv.2409.00088
33. Zhao, W., Deng, Z., Yadav, S., Yu, P.S.: Heterogeneous knowledge grounding for medical question answering with retrieval augmented large language model. In: Companion Proceedings of the ACM Web Conference 2024 (WWW '24). pp. 1590–1594. ACM (2024). https://doi.org/10.1145/3589335.3651941
34. Zhou, H., Boovaraghavan, S., Goel, M., Agarwal, Y.: On-device speech filtering for privacy-preserving acoustic activity recognition. In: Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom '24). pp. 1802–1804 (2024). https://doi.org/10.1145/3636534.3698865